Laboratorio introductorio de Python

Listas, bucles y condicionales · Guia practica para principiantes

Objetivo: comprender como guardar varios datos en listas, recorrerlos con for, repetir acciones con while y combinar bucles con condiciones. Este laboratorio se realiza antes del Laboratorio Puente y del Laboratorio 3 de archivos.

Preparacion

1. Abre la carpeta **ejercicios** en VS Code. 2. Verifica Python con **python --version** (o **py --version** en Windows). 3. Ejecuta un archivo con **python ejercicios/01_listas_basicas.py** desde la carpeta del proyecto, o utiliza el boton Ejecutar de VS Code. 4. Lee los comentarios **# TAREA**, modifica el programa y prueba diferentes valores.

Conceptos esenciales

Lista: agrupa valores en una variable, por ejemplo `nombres = ["Ana", "Luis"]`. Las posiciones comienzan en 0. En Python las listas son similares a los arreglos de otros lenguajes, pero son mas flexibles.

for: recorre elementos o una secuencia de numeros. **while:** repite un bloque mientras su condicion sea verdadera. **if:** decide que hacer dentro de un bucle. **Contador:** aumenta de uno en uno. **Acumulador:** suma valores.

```
productos = ["Mouse", "Teclado"]
for producto in productos:
    print(producto)

intentos = 0
while intentos < 3:
    print("Intento", intentos + 1)
    intentos += 1
```

Reglas de practica

Trabaja solo con los archivos de este paquete. No se requieren datos reales, credenciales ni acceso a sistemas de la empresa. Los ejercicios no leen, mueven ni eliminan archivos del equipo. Ejecuta, observa, modifica y vuelve a ejecutar; no hay informes ni evaluaciones.

1. Listas: crear y consultar

Abre **ejercicios/01_listas_basicas.py**. Ejecuta el codigo inicial, observa la salida y completa los comentarios indicados.

```
productos = ["Mouse", "Teclado", "Monitor"]
print("Lista completa:", productos)
print("Primer producto:", productos[0])
# TAREA 1: Mostrar el ultimo producto usando indice negativo.
# TAREA 2: Agregar "Webcam" a la lista y mostrarla.
# TAREA 3: Mostrar cuantos productos hay.
```

• TAREA 1: Mostrar el ultimo producto usando indice negativo.

• TAREA 2: Agregar "Webcam" a la lista y mostrarla.

• TAREA 3: Mostrar cuantos productos hay.

2. Modificar y eliminar elementos

Abre **ejercicios/02_editar_listas.py**. Ejecuta el codigo inicial, observa la salida y completa los comentarios indicados.

• TAREA 1: Agregar "morado".

• TAREA 2: Eliminar "rojo" usando remove().

• TAREA 3: Mostrar el resultado y su longitud.

3. Recorrer listas con for

Abre **ejercicios/03_for_productos.py**. Ejecuta el codigo inicial, observa la salida y completa los comentarios indicados.

```
productos = ["Laptop", "Mouse", "Audifonos", "Monitor"]
for producto in productos:
    print("Producto:", producto)
# TAREA 1: Mostrar los nombres en mayusculas con upper().
# TAREA 2: Mostrar cuantos productos se recorrieron usando un contador.
```

• TAREA 1: Mostrar los nombres en mayusculas con upper().

• TAREA 2: Mostrar cuantos productos se recorrieron usando un contador.

4. Repeticiones con range

Abre **ejercicios/04_range_y_repeticiones.py**. Ejecuta el codigo inicial, observa la salida y completa los comentarios indicados.

```
for numero in range(1, 6):
    print("Numero:", numero)
# TAREA 1: Mostrar los numeros del 1 al 10.
# TAREA 2: Mostrar solo los pares del 2 al 20 usando range(2, 21, 2).
# TAREA 3: Mostrar la tabla de multiplicar del 3 (del 1 al 10).
```

• TAREA 1: Mostrar los numeros del 1 al 10.

• TAREA 2: Mostrar solo los pares del 2 al 20 usando range(2, 21, 2).

• TAREA 3: Mostrar la tabla de multiplicar del 3 (del 1 al 10).

5. Combinar for e if

Abre **ejercicios/05_condiciones_en_for.py**. Ejecuta el codigo inicial, observa la salida y completa los comentarios indicados.

```
notas = [9, 6, 8, 4, 10]
for nota in notas:
    if nota >= 7:
        print(nota, "Aprobado")
# TAREA 1: Mostrar tambien las notas reprobadas con else.
# TAREA 2: Contar aprobados y reprobados.
```

• TAREA 1: Mostrar tambien las notas reprobadas con else.

• TAREA 2: Contar aprobados y reprobados.

• TAREA 3: Mostrar ambos totales.

6. Sumar valores de una lista

Abre **ejercicios/06_acumuladores.py**. Ejecuta el codigo inicial, observa la salida y completa los comentarios indicados.

```
precios = [12.5, 8.0, 20.0, 4.5]
total = 0
for precio in precios:
    total += precio
print("Total:", total)
# TAREA 1: Contar precios mayores que 10.
```

• TAREA 1: Contar precios mayores que 10.

• TAREA 2: Calcular el promedio usando total / len(precios).

• TAREA 3: Mostrar total y promedio con dos decimales.

7. Repetir con while

Abre **ejercicios/07_while_intentos.py**. Ejecuta el codigo inicial, observa la salida y completa los comentarios indicados.

```
clave_correcta = "python123"  # Clave ficticia para practicar
intentos = 0
while intentos < 3:
    clave = input("Escribe la clave de practica: ")
    intentos += 1
    if clave == clave_correcta:
```

• TAREA 1: Mostrar cuantos intentos se utilizaron.

• TAREA 2: Si no se acierta tras 3 intentos, mostrar "Intentos agotados".

8. Lista de compras interactiva

Abre **ejercicios/08_lista_compras.py**. Ejecuta el codigo inicial, observa la salida y completa los comentarios indicados.

• TAREA 1: Mostrar los productos numerados con enumerate(compras, start=1).

• TAREA 2: Agregar opcion 4 para buscar un producto (ignorar mayusculas).

• TAREA 3: Mostrar la cantidad de productos al salir.

9. Detective de errores

Abre **ejercicios/09_detective_errores.py**. Ejecuta el codigo inicial, observa la salida y completa los comentarios indicados.

• TAREA 1: Corregir el error de sintaxis.

• TAREA 2: Corregir el indice incorrecto.

• TAREA 3: Agregar un contador de frutas.

Este archivo no ejecutara hasta corregir sus errores intencionales. Usa los mensajes de Python como pistas.

10. Reto: inventario de cantidades

Abre **ejercicios/10_reto_desde_cero.py**. Ejecuta el codigo inicial, observa la salida y completa los comentarios indicados.

• RETO DESDE CERO (sin codigo inicial)

• 1. Recorre las listas usando range(len(productos)).

• 2. Muestra nombre y cantidad de cada producto.

• 3. Si cantidad es 0, muestra "Agotado".

• 4. Si cantidad esta entre 1 y 5, muestra "Stock bajo".

• 5. Si cantidad es mayor a 5, muestra "Disponible".

• 6. Calcula el total de unidades.

• EXTRA: Permite agregar un nuevo producto y su cantidad.

En este reto el archivo solo contiene instrucciones: escribe tu propio programa basandote en los ejercicios anteriores.

Antes de pasar al Laboratorio Puente

Comprueba en VS Code que puedes: crear y modificar listas; recorrerlas con for; repetir con while sin ciclos infinitos; combinar if con bucles; contar y sumar valores; y corregir errores de indentacion o indices. Luego podras practicar la clasificacion simulada de nombres de archivos.